Botler : AI-Powered Customer Engagement Platform

Creating a Scalable, Cost-Effective System to manage Inbound Dialing offering CPaaS

404_Sher_NotFound

Pranav Prakash
Priyansh Jain
BTech in Electronics Engineering

January 2026

This page intentionally left blank.

Acknowledgements

This project proposal would not have been possible without the support and resources provided by numerous individuals and open-source communities.

I would like to express my sincere gratitude to the DevBits team. The technical guidance provided in their "Helper Doc" was an invaluable resource that clarified complex implementation details and significantly streamlined the development process.

I am deeply indebted to the open-source community, whose robust tools form the backbone of this project. Specifically, I would like to acknowledge:

- **The Asterisk Community:** For providing the powerful telephony engine that serves as the foundation of this system.
- **The Pipecat Framework:** For the innovative orchestration layer that made handling real-time, bi-directional audio streams seamless.
- **The Creators of Faster-Whisper and Piper:** For their optimization of STT and TTS models, which enabled the low-latency performance required for a fluid conversational experience.

I also wish to thank the developers of the IPLeiria Theme, who made this LaTeXtemplate.

Preparation of this work was facilitated by the use of the *IPLeiria-Thesis* template.

Contents

1 Introduction to the Project: Motivation and Objectives **1**
1.1 Project Motivation 1
1.2 Project Objectives 1

2 Approach to the Problem **2**
2.1 Telephony Requirements 2
2.2 Real-Time Communication Requirements 2
2.3 Streaming-First Architecture 2
2.4 Contextual Intelligence 2
2.5 Post-Call Processing 3
2.6 Modular Design 3
2.7 Error Handling Strategy 3

3 Technical Selection & Justification **4**
3.1 Technology Stack 4
3.2 Why Asterisk over Twilio/FreePBX ? 5
3.3 Why Faster-Whisper over Whisper or alternatives? 5
3.4 Why Gemini over other alternatives? 5
3.5 Why Piper over other alternatives? 5
3.6 Why Silero VAD? 5
3.7 Pipecat 6

4 Minutes of Meeting Generator **8**
4.1 Recording conversation 8
4.2 Generating MoM from Recording 8

5 Additional planned features for Deployment (NOT FINAL) **9**
5.1 Inclusion of chan_mobile 9
5.2 OutBound Calling 9

This page intentionally left blank.

1

Introduction to the Project Motivation and Objectives

Botler is an AI-Powered Communications Platform as a Service (CPaaS) that aims to be a scalable and cost-effective system to manage Inbound Dialing using AI. It serves as a robotic butler, hence Botler. This chapter introduces its objectives and breaks down the problem it is meant to solve.

1.1 Project Motivation

Enterprises receive thousands of inbound calls daily but cannot afford to hire proportional human staff. A real estate company like DLF needs an AI-powered Communication Platform as a Service to automate customer engagement, contextually profile users, and generate actionable insights from call recordings.

1.2 Project Objectives

Botler is supposed to act as a robotic butler on call. It must act as both a smart receptionist and a smart secretary. Therefore, it is going to be implemented with 2 modules.

1. **Module A :** An AI Voice Agent that acts as a smart receptionist
   - Handles inbound calls with contextual awareness
   - Understands what the customer wants
   - Collects information from customers for profiling
2. **Module B :** An automated Minutes of Meeting (MoM) generator
   - Takes raw call recording as input
   - Transcribes the recording
   - Summarizes key discussion points
   - Extract action items for sales teams

2
Approach to the Problem

The challenge requires usage of Speech-to-Text (STT) component, Large Language Models (LLMs), Text-to-Speech (TTS) component, and of course, Telephony component. This chapter analyzes the requirements of each individual component and discusses the philosophy of system design.

2.1 Telephony Requirements

An asynchronous open source Private Branch Exchange (PBX) that can manage incoming calls and supports real-time bidirectional audio streaming.

2.2 Real-Time Communication Requirements

Real-time bidirectional audio streaming is required for processing input & outputting responses with low latency. WebSocket is the choice for streaming because it provides persistent, full-duplex communication channel over a single connection, reducing the latency and overhead associated with traditional HTTP request-response model.
Voice Activity Detection (VAD) is also needed to detect speech boundaries. To ensure effective communication low latency approaches are optimal.

2.3 Streaming-First Architecture

We have opted for streaming first architecture as it keeps the conversation flow human like and natural. It also achieves low latency as input is processed as soon as it is received.

2.4 Contextual Intelligence

The responses need to change as the conversation flows depending on the input from the customers, this requires contextual intelligence. Also contextual intelligence serves

a part in profiling the user and makes better informed decisions.

2.5 Post-Call Processing

After the call is over, the user needs to be profiled for the sales team. This requires summary of the call (Minutes of Meeting) and will be more accurate after the whole conversation is over as the final decision of the customer may vary throughout the conversation.

2.6 Modular Design

Modular design breaks code into independent, manageable units (modules) to enhance maintainability, scalability, and reusability, significantly reducing development complexity and debugging time. It improves readability, allows for team-based parallel development, and enables independent testing of components without affecting the overall system. This enables swapping VAD, STT, LLM, TTS when newer alternatives are available, improving scaling.

2.7 Error Handling Strategy

There may be invalid inputs such as coughs, or background noise or barge-ins throughout the conversation. There may also be system instability if the user says something unexpected. We propose using Voice Activity Detection to filer out background noise and also a mechanism to stop output when barge in occurs. We will also select an LLM that has guardrails and give it suitable prompts to prevent system instability due to unexpected conversation.

3

Technical Selection Justification for Selection

This chapter introduces the technological stack and the essentials for creating our project. Here we will explain the technological stack and provide justification for each chosen component. We will also compare the components with their respective alternatives.

3.1 Technology Stack

In Table 3.1 we have provided a summary of each component and why it was chosen. In further sections, we will explore why each Tool was chosen.

Table 3.1: *Tech Stack*

Component	Selected Tool	Why We Chose This
Telephony	Asterisk	Industry standard, open-source, self-hosted control
STT (Ears)	Faster-Whisper	Open-source, local processing
LLM (Brain)	Gemini API	Free tier
TTS (Mouth)	Piper	Offline, Sounds Natural
VAD	Silero VAD	High Accuracy, Lightweight
Framework	Pipecat	Open-source, Enterprise-grade

Note

The technology stack has been defined as per our initial draft and architecture of the project. As of now these are best tools for the job. However, if there are some design changes or a better tool is found that achieves our purpose, we may replace it. We will include the tech stack in the project README too.

3.2 Why Asterisk over Twilio/FreePBX ?

Twilio is an API provider and it is closed source. It charges per minute, per message, per phone number. If you have high-volume calls, the bills can become astronomical. Media flows through their servers, this means that highly regulated industries like banking, government will face privacy issues.
FreePBX is a GUI wrapper built on top of Asterisk. It is primarily designed for standard business phone systems like extensions, queues, IVRs.
It heavily relies on the traditional dialplan and configs, it is not optimized for our purpose and would require significant overrides that can get messy.
Asterisk on the other hand is a free and open source framework for building communications applications. It can be self-hosted and is highly customizable.
It added support for WebSocket channel driver option (chan_websocket) in September 2025 for easier media handling compared to AudioSocket or RTP. We are using **Linphone** (**open-source**) to configure a client as an SIP softphone.

3.3 Why Faster-Whisper over Whisper or alternatives?

Faster-Whisper is preferred over standard Whisper because it is a re-implementation via CTranslate2 providing faster performance than original OpenAI Whisper. There is no compromise in accuracy as it uses the same underlying Whisper models, ensuring quality of transcription remains the same.
It is also open-source and can be self-hosted, in contrast to, Deepgram or other alternatives. This reduces latency and also provides privacy.

3.4 Why Gemini over other alternatives?

Currently we are limited by the budget of the project and the hardware available to us. Thus, we cannot choose a self hosted LLM like LlaMa 3, Gemma 2, etc. Gemini provides a free API key which can be used as our LLM. We plan to make the LLM easily swappable (hence the modular approach), to possibly switch to a self-hosted LLM for scalability and actual deployment.

3.5 Why Piper over other alternatives?

Piper runs entirely locally, providing complete privacy, reducing latency and eliminating API key requirements.

3.6 Why Silero VAD?

Silero VAD was chosen because the model is very small and fast which is critical for real-time applications. It is free to use without requiring API keys or user registration.

3.7 Pipecat

Building a custom orchestration layer for real-time voice is very complex. To make a **custom FastAPI server** "enterprise-grade" we will need to solve :

- Modularity
- Latency Management
- Interruption Handling (barge-in)
- Scalability
- Jitter Buffering

Pipecat is an open source Python framework for building voice and multimodal AI bots that can see, hear, and speak in real-time.
The framework orchestrates AI services, network transports, and audio processing to enable ultra-low latency conversations that feel natural and responsive.
It is enterprise-grade and is recommended by Nvidia Blueprints and AWS blogs for building conversational AI. Pipecat already supports telephony and has docs on integrating Twilio, Telnyx, Plivo and Exotel WebSockets. Thus, we have a pathway to integrating Asterisk WebSockets.
If you are still not convinced, **Tail** is an official **terminal dashboard** for the Pipecat. It lets you monitor your Pipecat sessions in real time with logs, conversations, metrics, and audio levels all in one place. With Tail you can:

- Follow system logs in real time
- Track conversations as they happen
- Monitor user and agent audio levels
- Keep an eye on service metrics and usage
- Run locally as a pipeline runner or connect to a remote session

We have compared our 2 options in Table 3.2 below:

Table 3.2: *Comparison of Architecture Options: Pipecat vs. Custom Server.*

Feature	Option 1: Pipecat	Option 2: Custom Server
Modularity	**High.** Pipeline architecture allows you to swap STT/LLM/TTS providers (e.g., Deepgram ↔ OpenAI) with one line of code.	**Low.** You must manually write adapter classes for every API and manage their specific streaming protocols.
Latency	**Optimized.** Built-in handling for stream buffering and asynchronous frame processing to minimize "Time to First Byte" (TTFB).	**Variable.** You must manually optimize asyncio loops to prevent blocking; extremely prone to jitter without careful engineering.

Continued on the next page.

Table 3.2 continued from previous page.

Feature	Option 1: Pipecat	Option 2: Custom Server
Barge-in	**Native.** Includes sophisticated logic to stop TTS immediately when VAD detects speech (interruptibility).	**Manual.** You must write complex logic to flush audio buffers and cancel TTS streams instantly upon VAD triggers.
VAD	**Integrated.** Supports Silero VAD and others with tunable thresholds within the pipeline.	**Manual.** You must implement VAD on the incoming audio stream and sync it with the conversation state manually.
Scalability	**Container-Ready.** Designed to run as stateless worker processes (one bot per container/process).	**Manual.** You must design your own worker management system to handle concurrent websocket connections.

We have also included an Architecture overview of Botler Module A in Fig 3.1.

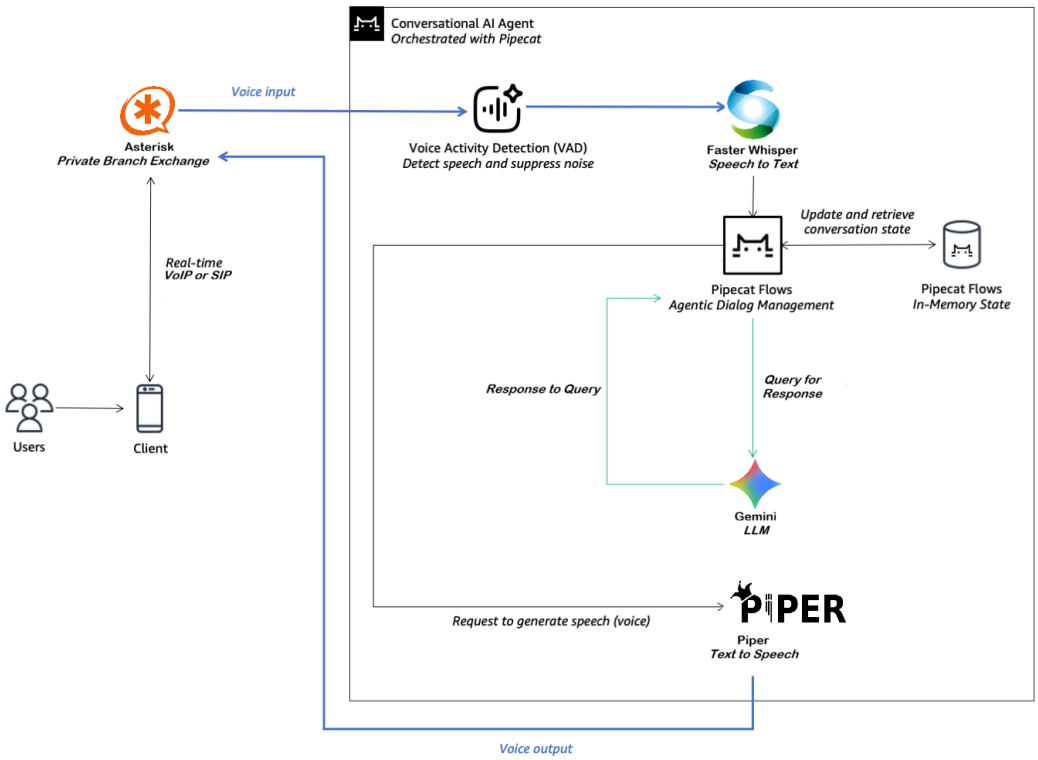

Figure 3.1: *Architecture overview of Botler Module A using Pipecat.*

4

Minutes of Meeting Generator Module B

Module B of Botler is going to be a Minutes of Meeting Generator. According to the Problem Statement it will take the raw audio file of the conversation, process it and output a structured "Minutes of Meeting" document. The system should list:

- Key discussion points
- Customer requirements identified
- Action items for the sales team

> **Note**
>
> The implementation of Module B is heavily dependent on how we implement Module A. This is because the conversation recording will be handled in the implementation of Module A. Here we are listing our initial ideas to handle Module B assuming that our Module A is implemented as described in Chapter 3.

4.1 Recording conversation

1. **Record using Asterisk :** Use MixMonitor in dialplan.
2. **Record using Pipecat :** Use AudioBufferProcessor.

4.2 Generating MoM from Recording

As compared to Module A, this seems basic. Take the call recording, process it through VAD, STT, LLM to generate a structured summary of the call. We can handle this using Pipecat.
Alternatively, we can maintain a summary and keep updating it while the call is active, eliminating the need for a recording.

5
Additional Features

Warning

This chapter is merely included to discuss possible feature additions. Their inclusion in the final submission is subject to time and development constraints. If the previously discussed modules A and B are successfully created before the deadline and work perfectly, we may work on the aforementioned features.

5.1 Inclusion of chan_mobile

chan_mobile allows the enterprise to use a personal mobile phone through bluetooth instead of an SIP or VoIP service. This means that the enterprise can connect a SIM provided by a regular operator (e.g.- Jio, Airtel) and anyone calling that number will be able to interface with Botler.

5.2 OutBound Calling

Using Pipecat and Asterisk to make outbound AI calls to customers.

This page intentionally left blank.